

Three-Tier Application using Docker & Docker Compose

Final Project Documentation – AWS Cloud & DevOps Course

Project	Django Notes App – Three-Tier Architecture on Docker
Author	Jotiba Bhate
Course	AWS Cloud & DevOps
Environment	Killercoda Ubuntu Playground
Tools	Docker, Docker Compose, Nginx, Django (Python 3.9), MySQL, Git, Vim
Date	6 October 2026

Summary: This project deploys a Django Notes web application as a three-tier architecture (Web tier, Application tier, Database tier) using Docker containers managed with Docker Compose. A single `docker-compose up -d` command builds the images, creates a private network and starts all three containers.

Table of Contents

1. Project Overview
2. Architecture
3. Technology Stack
4. Container Details
5. Implementation Steps
6. Output and Verification
7. Troubleshooting
8. Useful Commands
9. Key Learnings
10. Conclusion

1. Project Overview

The objective of this project is to containerize a Django Notes application and run it as a production-style three-tier setup. Each tier runs in its own Docker container, so every component can be built, started, updated and scaled independently.

- **Objective:** Deploy a multi-container application using Docker Compose.
- **Application:** Django Notes App (create and view notes in the browser).
- **Source code:** Cloned from a public GitHub repository (django-notes-app).
- **Outcome:** Application running successfully; notes can be added and displayed.

2. Architecture

The application follows the classic three-tier model. The browser talks to Nginx, Nginx forwards requests to Django, and Django reads and writes data in MySQL. All containers share one user-defined Docker network and reach each other by service name.

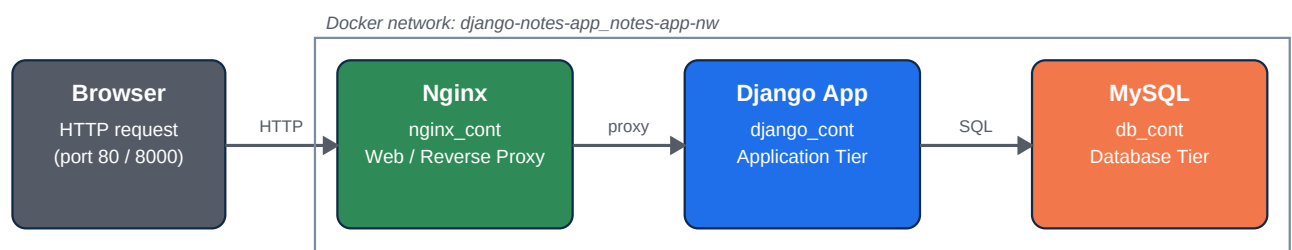


Figure 1: Three-tier architecture of the Django Notes application

Tier	Component	Responsibility
Web tier	Nginx	Receives HTTP requests on port 80 and acts as web server / reverse proxy
Application tier	Django (Python)	Runs business logic and serves the Notes application on port 8000
Database tier	MySQL	Stores notes data persistently; listens on port 3306 (internal)

3. Technology Stack

Technology	Version / Image	Purpose
Docker	Docker Engine	Containerization platform
Docker Compose	docker-compose	Defines and runs multi-container apps
Python	python:3.9 (base image)	Runtime for Django application
Nginx	nginx:1.23.3-alpine	Web server / reverse proxy
MySQL	mysql:latest	Relational database
Git	git	Clone project source code
Vim	vim	Edit Dockerfile, docker-compose.yml and .env

4. Container Details

Service	Container name	Image	Port mapping
db	db_cont	mysql:latest	3306 (internal)
django_app	django_cont	django_app (built from Dockerfile)	8000 : 8000
nginx	nginx_cont	nginx (built from nginx/Dockerfile)	80 : 80

Network: django-notes-app_notes-app-nw (bridge driver, created automatically by Docker Compose).

Django Dockerfile (build stages observed in build log):

```
FROM python:3.9
WORKDIR /app/backend
COPY requirements.txt /app/backend
RUN apt-get update && apt-get upgrade -y \
    && apt-get install -y gcc default-libmysqlclient-dev pkg-config \
    && rm -rf /var/lib/apt/lists/*
RUN pip install mysqlclient
RUN pip install --no-cache-dir -r requirements.txt
COPY . /app/backend
```

Note: the system packages (gcc, libmysqlclient-dev, pkg-config) are required to compile the `mysqlclient` Python driver that connects Django to MySQL.

5. Implementation Steps

Step 1: Create the project directory

```
mkdir docker-project
cd docker-project/
```

Step 2: Clone the source code from GitHub

```
git clone https://github.com/thawaresameer715-cyber/django-notes-app.git
cd django-notes-app/
ls
```

The repository contains: Dockerfile, Jenkinsfile, Procfile, README.md, api, db.sqlite3, docker-compose.yml, manage.py, mynotes, nginx, notesapp, requirements.txt and staticfiles.

Step 3: Configure Dockerfile, docker-compose.yml and .env

```
vim Dockerfile
vim docker-compose.yml
vim .env
```

- **Dockerfile** – defines how the Django image is built.
- **docker-compose.yml** – defines the three services (db, django_app, nginx), the network and port mappings.
- **.env** – stores environment variables such as database credentials, kept out of the code.

Step 4: Check existing Docker networks

```
docker network ls
```

Initially only the default networks (bridge, host, none) exist.

Step 5: Build images and start all containers

```
docker-compose up -d
```

This single command performs the following actions:

- Creates the network `django-notes-app_notes-app-nw`.
- Pulls the `mysql:latest` image for the database.
- Builds the `django_app` image from the Dockerfile (took about 275 seconds on first build).
- Builds the `nginx` image (based on `nginx:1.23.3-alpine`).
- Creates and starts `db_cont`, `django_cont` and `nginx_cont` in detached mode.

```

root@ubuntu:~# ls
filesystem
root@ubuntu:~# mkdir docker-project
root@ubuntu:~# cd docker-project/
root@ubuntu:~/docker-project# git clone https://github.com/thawaresameer715-cyber/django-notes-app.git
Cloning into 'django-notes-app'...
remote: Enumerating objects: 408, done.
remote: Total 408 (delta 0), reused 0 (delta 0), pack-reused 408 (from 1)
Receiving objects: 100% (408/408), 1.68 MiB | 8.02 MiB/s, done.
Resolving deltas: 100% (102/102), done.
root@ubuntu:~/docker-project# ls
django-notes-app
root@ubuntu:~/docker-project# cd django-notes-app/
root@ubuntu:~/docker-project/django-notes-app# ls
Dockerfile Jenkinsfile Procfile README.md api db.sqlite3 docker-compose.yml manage.py mynotes nginx notesapp requirements.txt staticfiles
root@ubuntu:~/docker-project/django-notes-app# vim Dockerfile
root@ubuntu:~/docker-project/django-notes-app# vim docker-compose.yml
root@ubuntu:~/docker-project/django-notes-app# vim .env
root@ubuntu:~/docker-project/django-notes-app# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
ba2915173431        bridge              bridge              local
2accf8623d92        host                host                local
3bc04f36d963        none                null                local
root@ubuntu:~/docker-project/django-notes-app# docker-compose up -d
Creating network "django-notes-app_notes-app-nw" with the default driver
Pulling db (mysql):...
latest: Pulling from library/mysql
0da02031b810: Pull complete
661f04f054c4: Pull complete
1441bc0931d1: Pull complete
a67ddc05b7f8: Pull complete
20b7464ce26b: Pull complete
343c451c026a: Pull complete
330a9fb3841e: Pull complete
a95bd78f8620: Pull complete
f37786bee968: Pull complete
6bdafce1e6fb: Pull complete
Digest: sha256:9d48c42f8341068f199116dfc9b19b607c99765b5c61e549a548a43033471a4
Status: Downloaded newer image for mysql:latest
Building django_app
[+] Building 275.9s (12/12) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 476B
=> [internal] load metadata for docker.io/library/python:3.9
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/7] FROM docker.io/library/python:3.9@sha256:da5ee29682d12a6649f51c8d6f15b87deb3e6c524b923c41d0cb3304d07c913
=> => resolve docker.io/library/python:3.9@sha256:da5ee29682d12a6649f51c8d6f15b87deb3e6c524b923c41d0cb3304d07c913
=> => sha256:da5ee29682d12a6649f51c8d6f15b87deb3e6c524b923c41d0cb3304d07c913 10.30kB / 10.30kB
=> => sha256:d6ca7d9522a172c424721d3509ee12079f7864a742b6adf1eeb66b6c405307ee 2.32kB / 2.32kB
=> => sha256:bcd3da5974912584a81ed86fd944ab5fba9093ff1c9a0b0ed18349f9a69e4762 6.23kB / 6.23kB

```

Figure 2: Cloning the repository and running docker-compose up -d (MySQL image pull and Django build)

```

=> => extracting sha256:795dbedde24d2c72dafd2b71fe36643552e56859c0e29cd095ed54b825fb2a2 4.5s
=> => sha256:081cf923272c30c6072c6ff1617d9072e03ab2a90a431951d325d45e296962b 6.10MB / 6.10MB
=> => sha256:c9723aa529b03c40e66d0aee927a410b4719528ab865af6e0bac1b7c9b10829e 20.37MB / 20.37MB
=> => sha256:91c91cf1d23f4edf4280a8fe935f14340fec43a7a3576149a7cfcf70c2f9b 2508 / 2508
=> => extracting sha256:89d573bf42b377ce6a5a0451c15388849686f4a058ef8d8999f3b014daeb5b55 0.9s
=> => extracting sha256:62af12f2c1c406a9af1d1020a9380a2c442aa84f44452b7b0c8148ff1b3 2.8s
=> => extracting sha256:79d6b48d8a62418bf22e70553e438c67894c72a319d76b30aaf5c331b6924 8.8s
=> => extracting sha256:081cf923272c30c6072c6ff1617d9072e03ab2a90a431951d325d45e296962b 0.4s
=> => extracting sha256:c9723aa529b03c40e66d0aee927a410b4719528ab865af6e0bac1b7c9b10829e 0.9s
=> => extracting sha256:91c91cf1d23f4edf4280a8fe935f14340fec43a7a3576149a7cfcf70c2f9b 0.0s
=> [internal] load build context
=> => transferring context: 8.18MB
=> [2/7] WORKDIR /app/backend
=> [3/7] COPY requirements.txt /app/backend
=> [4/7] RUN apt-get update && apt-get upgrade -y && apt-get install -y gcc default-libmysqlclient-dev pkg-config && rm -rf /var/lib/apt/lists/* 230.6s
=> [5/7] RUN pip install mysqlclient
=> [6/7] RUN pip install --no-cache-dir -r requirements.txt
=> [7/7] COPY . /app/backend
=> exporting to image
=> => exporting layers
=> => writing image sha256:98bcd4f5d64d74e17d8e18bce5d1956174600c7e1603dc3fe3cca2b0be4c4b9a
=> => naming to docker.io/library/django_app
WARNING: Image for service django_app was built because it did not already exist. To rebuild this image you must use `docker-compose build` or `docker-compose up --build`.
Building nginx
[+] Building 5.9s (7/7) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 113B
=> [internal] load metadata for docker.io/library/nginx:1.23.3-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 411B
=> [1/2] FROM docker.io/library/nginx:1.23.3-alpine@sha256:6318314189b40e73145a48060bffa74783a116c34cc7241532d0d94198fb2c9629
=> => resolve docker.io/library/nginx:1.23.3-alpine@sha256:6318314189b40e73145a48060bffa74783a116c34cc7241532d0d94198fb2c9629
=> => sha256:3eb308b1387e9f2a49cb6e5e18db016e33d62c37ea0e9be2339e9f0b3e26170 1.78kB / 1.78kB
=> => sha256:2bc7edbc3cf2fce630a95d0586c48cd248e5df37df5b1244728a5c8c91bcefe0 16.35kB / 16.35kB
=> => sha256:6318314189b40e73145a48060bffa74783a116c34cc7241532d0d94198fb2c9629 1.65kB / 1.65kB
=> => sha256:63b65145d645c1250c391b2d16e6e53b3747c295ca8ba2fcb6b0cf064a4dc21c 3.37MB / 3.37MB
=> => sha256:8c7e1f96938084ba511a6c1f0d9fae406520605d6646b040927dd4bc14d866 1.80MB / 1.80MB
=> => sha256:86c5246c96dbb77b446cfbd8caaea7d71bbac064836e7eb5107e047da5cb6ead 627B / 627B
=> => extracting sha256:63b65145d645c1250c391b2d16e6e53b3747c295ca8ba2fcb6b0cf064a4dc21c 0.1s
=> => sha256:b874033c43fb2c911c74a6f9dc209b76bb4131c6990d3af5e00c1b6a927f6efe 956B / 956B
=> => sha256:db1551bd73f2a91dc74c53156da4d2d38074f14aba106e534ca32c6e49a0c3 772B / 772B
=> => extracting sha256:8c7e1f96938084ba511a6c1f0d9fae406520605d6646b040927dd4bc14d866 0.3s
=> => sha256:0d4f6b3f3de69252af87911eaab1781b43161bafdf3c4b9514a2ebce96a0cd5 1.40kB / 1.40kB
=> => sha256:2a41f256c40f00d75b71a31444da307b5a1884c3af9fdac91504acfc1141c6a 11.40MB / 11.40MB
=> => extracting sha256:86c5246c96dbb77b446cfbd8caaea7d71bbac064836e7eb5107e047da5cb6ead 0.0s
=> => extracting sha256:b874033c43fb2c911c74a6f9dc209b76bb4131c6990d3af5e00c1b6a927f6efe 0.0s
=> => extracting sha256:db1551bd73f2a91dc74c53156da4d2d38074f14aba106e534ca32c6e49a0c3 0.0s
=> => extracting sha256:0d4f6b3f3de69252af87911eaab1781b43161bafdf3c4b9514a2ebce96a0cd5 0.0s
=> => extracting sha256:2a41f256c40f00d75b71a31444da307b5a1884c3af9fdac91504acfc1141c6a 0.4s

```

Figure 3: Django and Nginx image build logs

```

=> exporting to image                                0.5s
=> exporting layers                                  0.3s
=> writing image sha256:d62604a69a4221fed18cbcd601c85036d1630d588a7c34f4caa3c355c19bb109 0.0s
=> naming to docker.io/library/nginx                 0.1s
WARNING: Image for service nginx was built because it did not already exist. To rebuild this image you must use `docker-compose build` or `docker-compose up --build`.
Creating db_cont ... done
Creating django_cont ... done
Creating nginx_cont ... done
root@ubuntu:~/docker-project/django-notes-app$ docker build -t notes-app .
[+] Building 10.4s (12/12) FINISHED
=> [internal] load build definition from Dockerfile
=> [internal] load metadata for docker.io/library/python:3.9
=> [internal] load .dockerignore
=> [internal] load context: 2B
=> [1/7] FROM docker.io/library/python:3.9@sha256:da5ae29682d12a6649f51c8d6f15b87deb3e6c524b923c41d0cb3304d07c913
=> [internal] load build context
=> [internal] load metadata for docker.io/library/python:3.9
=> [2/7] FROM docker.io/library/python:3.9@sha256:da5ae29682d12a6649f51c8d6f15b87deb3e6c524b923c41d0cb3304d07c913
=> [internal] load build context
=> [3/7] COPY requirements.txt /app/backend
=> [4/7] RUN apt-get update && apt-get upgrade -y && apt-get install -y gcc default-libmysqlclient-dev pkg-config && rm -rf /var/lib/apt/lists/*
=> [5/7] RUN pip install mysqlclient
=> [6/7] RUN pip install --no-cache-dir -r requirements.txt
=> [7/7] COPY . /app/backend
=> exporting to image
=> exporting layers
=> writing image sha256:87a667d3af7b575389b6239c9e9060811da8d4f43e273a59c7277bd06892ff68
=> naming to docker.io/library/notes-app
root@ubuntu:~/docker-project/django-notes-app$ docker run -d -p 8000:8000 notes-app:latest
4b66c213f88b1e2447109cb0541739c09c64a390e109f7f5af6b8c0ffa7445f
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint pedantic_mendelev (709f12e2672a0c40c81cd9a95c35da85a
dd2d8ca66862f2d593e5fb6ca2432c1): Bind for 0.0.0.0:8000 failed: port is already allocated

Run 'docker run --help' for more information
root@ubuntu:~/docker-project/django-notes-app$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
4b66c213f88b   notes-app:late "python3"               24 seconds ago Up 24 seconds Created                               pedantic_mendelev
f378ea6acc08   nginx          "/docker-entrypoint..." About a minute ago Up About a minute Up 80->80/tcp                        nginx_cont
0fae8a782f8b   django_app     "sh -c 'python manag..." About a minute ago Up 57 seconds (healthy) Up 8000->8000/tcp, [::]:8000->8000/tcp django_cont
c371df0b25fc   mysql          "docker-entrypoint.s..." About a minute ago Up About a minute (healthy) 3306/tcp, 33060/tcp                db_cont

```

Figure 4: All three containers created; container status shown with `docker ps -a`

6. Output and Verification

After the containers started, `docker ps -a` confirmed that all services were running and healthy:

Container	Status	Ports
nginx_cont	Up	0.0.0.0:80->80/tcp
django_cont	Up (healthy)	0.0.0.0:8000->8000/tcp
db_cont	Up (healthy)	3306/tcp, 33060/tcp

The application was opened in the browser on port 8000. The **My Notes** page loaded successfully, and two notes were added ("hi i am jotiba bhate" and "my first docker project"), proving that the Django application, the MySQL database and the network between them work correctly.

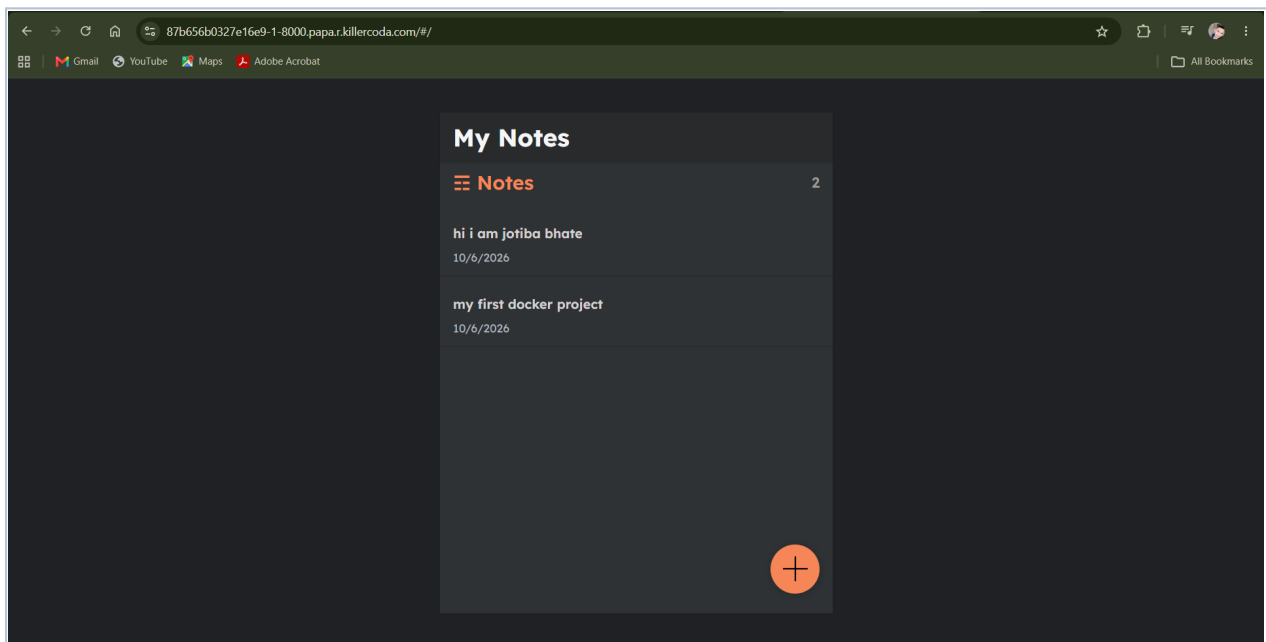


Figure 5: Django Notes application running in the browser with notes saved

7. Troubleshooting

Error: port is already allocated

```
docker build -t notes-app .
docker run -d -p 8000:8000 notes-app:latest

Error: Bind for 0.0.0.0:8000 failed: port is already allocated
```

Item	Details
Cause	Docker Compose had already started <code>django_cont</code> , which is using host port 8000. A second container cannot bind to the same host port.
Result	The extra container (<code>notes-app</code>) was only created, not started, and is visible as <code>Created</code> in <code>docker ps -a</code> .
Solution	Not needed for a Compose-based deployment. Either use the Compose stack, or map a different host port (for example <code>-p 8001:8000</code>), or stop the existing container first.
Cleanup	<code>docker rm <container_id></code> removes the unused container.

Other note: Compose printed a warning that images were built because they did not exist. To rebuild after code changes, use `docker-compose up -d --build`.

8. Useful Commands

Command	Purpose
<code>docker-compose up -d</code>	Build (if needed) and start all services in background
<code>docker-compose up -d --build</code>	Rebuild images and restart services

Command	Purpose
<code>docker-compose down</code>	Stop and remove containers and network
<code>docker-compose logs -f</code>	Follow logs of all services
<code>docker ps -a</code>	List all containers with status
<code>docker images</code>	List local images
<code>docker network ls</code>	List Docker networks
<code>docker exec -it django_cont bash</code>	Open a shell inside the Django container

9. Key Learnings

- Understood the three-tier architecture and how each tier maps to a container.
- Wrote and used a Dockerfile to build a custom Django image.
- Used Docker Compose to define and run a multi-container application with one command.
- Learned how Docker networking lets containers communicate using service names.
- Used an `.env` file to keep configuration separate from code.
- Diagnosed and understood a host port conflict error.

10. Conclusion

The Django Notes application was successfully deployed as a three-tier architecture using Docker and Docker Compose. Nginx, Django and MySQL run in separate, isolated containers on a shared network, and the application was verified in the browser. This project demonstrates practical skills in containerization, multi-container orchestration, networking and troubleshooting, which are core skills for Cloud and DevOps roles.

Prepared by: Jotiba Bhate | AWS Cloud & DevOps